

Rapport — Jour 4

Calculabilité : machine de Turing & machine universelle

Binôme : DOSSOU Persévérance & AGBOGBA Silas • Modules :
formlang/turing.py, utm.py — App : apps/mtu

A. Réponses théoriques

Q4.1 (0,5) — encodage $\langle M \rangle$

Schéma. On numérote (implicitement, par leur nom de chaîne) les états Q et les symboles Γ de M . La table δ est linéarisée comme un dictionnaire de quintuplets $(q, a) \mapsto (q', b, d)$ avec $d \in \{L, R, S\}$, sérialisé en JSON (clé textuelle "q|a" $\rightarrow$ liste $[q', b, d]$), auquel on ajoute l'état initial, l'ensemble des états acceptants, le symbole blanc et l'ensemble des états de rejet. Le tout est produit par `json.dumps(..., sort_keys=True)` : $\langle M \rangle$ est donc une **chaîne de caractères unique**.

Injectivité. Deux machines distinctes (transitions, état initial, acceptants, blanc ou rejet différents) produisent deux dictionnaires Python distincts, donc deux chaînes JSON distinctes (JSON est une sérialisation déterministe ici grâce à `sort_keys=True` : pas d'ambiguïté d'ordre) : `encode` est injective.

Décodabilité. `decode` est la réciproque exacte : elle relit le JSON, sépare chaque clé "q|a" sur le premier "|" pour retrouver (q, a) , et reconstruit un `TuringMachine` avec exactement les mêmes champs. Le *piège* de l'énoncé (« sortie non triée $\Rightarrow$ round-trip non déterministe ») est neutralisé par le tri systématique des clés avant sérialisation : `decode(encode(M))` redonne une machine strictement équivalente à M pour tout mot d'entrée.

Q4.2 (0,5) — la machine universelle

Description. U prend en entrée $\langle M \rangle \# w$ (ici, deux arguments séparés : la description $\langle M \rangle$ et le mot w). U **décode** $\langle M \rangle$ (trouve $Q, \Gamma, \delta, q_0, F$), puis **simule** pas à pas M sur w en utilisant la même boucle générique `TuringMachine.run` — U ne réécrit aucune logique propre à M , elle délègue entièrement l'exécution.

Invariant maintenu. À chaque étape t de la simulation, le ruban de U (restreint à la zone utile) est exactement le ruban qu'aurait M après t pas d'exécution directe sur w , et l'état courant simulé correspond à l'état réel de M à ce même pas.

Argument de correction (récurrence sur le nombre de pas t). *Base* ($t = 0$) : le ruban initial de la simulation est w , comme celui de $M(w)$ à $t = 0$ — vrai par construction. *Hérédité* : si l'invariant est vrai au pas t (même état, même ruban), alors U lit le même symbole que M lirait, applique la même transition $\delta(q, a) = (q', b, d)$ (car δ décodée est exactement celle de M), écrit le même symbole, déplace la tête de la même façon $\Rightarrow$ l'invariant est vrai au pas $t+1$. Par récurrence, U s'arrête en acceptant w ssi M accepte w , avec le même ruban final — exactement ce que vérifie `test_utm_simule_comme_la_machine`.

Q4.3 (0,5) — surcoût (sourcé)

Simuler une machine multi-ruban par une machine à un seul ruban entraîne un ralentissement au plus **quadratique** dans le nombre de pas ; à l'inverse, simuler M par U (programme enregistré) coûte $O(t \cdot |\langle M \rangle|)$ par pas simulé, où t est le nombre de pas de M (chaque pas de M nécessite de parcourir $\langle M \rangle$ pour retrouver la transition applicable). Hennie et Stearns (*Two-Tape Simulation of Multitape Turing Machines*, J. ACM 13(4), 1966) montrent qu'un ralentissement seulement **logarithmique** (plutôt que quadratique) est possible entre certaines classes de machines multi-ruban ; aucune borne n'est ici affirmée « de mémoire » sans cette référence, conformément à la charte d'honnêteté intellectuelle de l'énoncé.

Q4.4 (0,5) — indécidabilité

Réduction depuis halt. Le problème HALT (« M s'arrête-t-elle sur w ? ») est indécidable (Turing, 1936). Supposons par l'absurde qu'il existe un décideur D pour « U s'arrête sur $\langle M \rangle \# w$ ». Comme U simule fidèlement M sur w pas à pas (Q4.2), U s'arrête sur $\langle M \rangle \# w$ si et seulement si M s'arrête sur w . On pourrait alors décider HALT en construisant $\langle M \rangle$ (calculable, Q4.1) et en appelant D dessus — contradiction avec l'indécidabilité de HALT. Donc « U s'arrête sur $\langle M \rangle \# w$ » est indécidable.

Commentaire. « Universel $\neq$ omniscient » : U peut exécuter *n'importe quelle* machine décrite, mais elle ne peut pas, en général, prédire à l'avance si cette exécution va terminer. L'universalité (simuler tout calcul) et la décidabilité (prédire l'issue de tout calcul) sont deux propriétés distinctes ; le projet gagne la première au jour 4 mais perd définitivement la seconde — c'est la leçon annoncée dès la présentation générale de l'énoncé.

B. Exercices de programmation

E4.1

`TuringMachine.run (formlang/turing.py)` : ruban représenté par un dict {position $\rightarrow$ symbole} (bi-infini, pas de limite de taille), tête = un entier `pos`. À chaque pas : lecture de `tape.get(pos, blank)`, recherche de (q, a) dans la table ; absence de transition $\Rightarrow$ arrêt immédiat (mot rejeté si l'état courant n'est pas acceptant, **sans lever d'exception** — piège de l'énoncé). Un compteur `max_steps` (option `trace=True`

pour obtenir la trace complète pas-à-pas) garantit qu'une boucle infinie lève une erreur explicite plutôt que de bloquer indéfiniment. *Test test_trace_disponible : passé.*

E4.2

encode/decode (formlang/utm.py) : voir Q4.1; UniversalTM.run décode puis délègue à TuringMachine.run. *Tests test_round_trip_encodage, test_utm_simule_comme_la_machine : passés.*

E4.3

Tables ADD/SUB (apps/mtu/machines.py) et Calculatrice (apps/mtu/calculator.py) :

- **ADD** : 3 états. On parcourt n (état q_0), le $+$ est **fusionné** en 1 (état q_1), on parcourt m jusqu'au blanc, demi-tour, et on **retire un 1** (compensation exacte du $+$ converti) avant d'accepter — trace conforme à l'exemple de l'énoncé ($2 + 1 = 3$ sur $11+1$).
- **SUB** : appariement par marquage. On cherche un 1 non marqué dans m (marqué X , de gauche à droite, états q_1/q_2), *apparié* à un 1 non barré de n , pris le **plus à droite** (barré Y , états q_3/q_4) — ce choix « droite $\rightarrow$ gauche » garantit que les 1 restants (la réponse) forment un bloc **contigu** à gauche, sans trou. On répète jusqu'à épuiser m (nettoyage q_5/q_6 : $X, Y, -$ effacés, les 1 non barrés restent) ou jusqu'à épuiser n avant m (état q_zero : $m > n$, tout le ruban est effacé, résultat 0). Les trois points délicats de l'énoncé sont traités : gérer ses propres marques (q_1 saute les X , q_3 saute les Y), barrage droite $\rightarrow$ gauche pour la contiguïté, fin de m détectée par un **blanc** et non un compteur.
- Calculatrice.addition/soustraction appellent directement ADD/SUB (aucune nouvelle boucle de MT); multiplication est m additions *composées*; division est une suite de soustractions composées comptant le quotient; chaîner applique une liste d'opérations séquentiellement.

Tests test_add_sub_machines_directes, test_calculatrice_exhaustif (49 couples $(n, m) \in \{0, \dots, 6\}^2$), test_chainage, test_division_par_zero : passés.

E4.4

UniversalInterpreter.run(machine, word) encode machine puis délègue à UniversalTM (apps/mtu/interpreter.py); addition_via_utm/soustraction_via_utm construisent le mot d'entrée unaire puis passent par l'interpréteur universel — aucune boucle de ruban n'est réécrite dans apps/mtu, conformément à la règle « gate » du jour. *Test test_interpreteur_universel : passé.*

Sortie de référence obtenue

```
$ pytest -q tests/test_turing.py tests/test_calc.py
8 passed
```

(Sur l'ensemble du dépôt : 25 passed, 3 failed — les 3 échecs restants, test_nerode_trois_classes, test_pipeline_word, test_pipeline_morpho, relèvent tous du Jour 5, non encore traité.)